

Projet No2 Intelligence Artificielle #1

Techniques de programmation

Par

Samira Lehlou

Michel Paquin

420-004-XX

Groupe 12504

Table des matières

Programmation Linéaire	3
Problème : trouver le montant optimal	3
Démarche théorique :	3
Programmation Python :	3
Programmation Standard	3
Programmation avec PuLP	4
Apprentissage et analyse des résultats	5
Temps algorithmique (Grand O)	6
1- Trois boucles imbriquées une à l'intérieur de l'autre	6
2- Une variable directement utilisée	6
3- Une recherche dans un arbre binaire	6
Récurtivité Simple	7
Code Python :	7
Résultats:	7
Notions d'apprentissage et élaboration des résultats :	7
Programmation Dynamique1	9
Code Python :	9
Résultats :	9
Analyse des résultats :	9
Test de Performance	11
Code Python	11
Résultat	11
Analyse des résultats	12

Programmation Linéaire

Problème : trouver le montant optimal

Nicki a deux emplois à temps partiel. Elle ne veut pas travailler plus de 12 heures et ne veut pas passer plus de 16 heures de préparation par semaine. Le travail no 1 rapporte \$40 de l'heure et demande 2 heures de préparation par heure de travail. Le travail no 2 rapporte \$30 de l'heure mais ne nécessite qu'une seule heure de préparation par heure de travail.

Démarche théorique :

Prendre le travail le plus payant et regarder quelles sont les combinaisons possibles :

Heures Emploi #1	Heures Emploi #1																			
	0		1		2		3		4		5		6		7		8		9	
	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep	\$	Prep
0	0	0	40	2	80	4	120	6	160	8	200	10	240	12	280	14	320	16	360	18
1	30	1	70	3	110	5	150	7	190	9	230	11	270	13	310	15	350	17	390	19
2	60	2	100	4	140	6	180	8	220	10	260	12	300	14	340	16	380	18	420	20
3	90	3	130	5	170	7	210	9	250	11	290	13	330	15	370	17	410	19	450	21
4	120	4	160	6	200	8	240	10	280	12	320	14	360	16	400	18	440	20	480	22
5	150	5	190	7	230	9	270	11	310	13	350	15	390	17	430	19	470	21	510	23
6	180	6	220	8	260	10	300	12	340	14	380	16	420	18	460	20	500	22	540	24
7	210	7	250	9	290	11	330	13	370	15	410	17	450	19	490	21	530	23	570	25
8	240	8	280	10	320	12	360	14	400	16	440	18	480	20	520	22	560	24	600	26
9	270	9	310	11	350	13	390	15	430	17	470	19	510	21	550	23	590	25	630	27
10	300	10	340	12	380	14	420	16	460	18	500	20	540	22	580	24	620	26	660	28
11	330	11	370	13	410	15	450	17	490	19	530	21	570	23	610	25	650	27	690	29
12	360	12	400	14	440	16	480	18	520	20	560	22	600	24	640	26	680	28	720	30

La combinaison la plus payante est 4 heures de l'emploi no 1 et 8 heures de l'emploi no2 : 12 heures par semaine, un total de \$400 et 16 heures de préparation.

Programmation Python :

Nous avons résolu le problème avec une programmation standard et avec PuLP

Programmation Standard

```
#Programmation Standard
Salaire_Optimum = 0
H1 = 0
H2 = 0
preparation_finale = 0

for Heures1 in range(12, 0, -1):
    for Heures2 in range((12 - Heures1), 0, -1):
        Preparation = (Heures1 * 2) + Heures2
```

```

    if Preparation <= 16:
        Salaire = (Heures1 * 40) + (Heures2 * 30)
        if Salaire > Salaire_Optimum:
            Salaire_Optimum = Salaire
            H1 = Heures1
            H2 = Heures2
            preparation_finale = Preparation

print ('Résultats Programmation Standard:')
print('Salaire Optimum : $', Salaire_Optimum)
print("Nombre d'heures Emploi à $40 :", H1)
print("Nombre d'heures Emploi à $30 :", H2)
print("Préparation :", str(preparation_finale))

```

Résultats Programmation Standard:

Salaire Optimum : \$ 400

Nombre d'heures Emploi à \$40 : 4

Nombre d'heures Emploi à \$30 : 8

Préparation : 16

Programmation avec PuLP

```

problème : LpProblem = LpProblem('Salaire', LpMaximize)
# Définition des variables
h1 = LpVariable('h1', 0, 12, LpInteger)
h2 = LpVariable('h2', 0, 12, LpInteger)
# définition des Contraintes
problème += (h1 + h2) <= 12
problème += ((h1*2) + h2) <= 16
#définition de l'objectif
problème += (h1*40) + (h2*30)
print('Résultat ave PuLP')
# On affiche le résultat
print(LpStatus[problème.solve(PULP_CBC_CMD(msg=False))])
#Show the final value of the optimal objective function
print(f"Salaire total : $ {problème.objective.value()}")

```

Résultat ave PuLP

Optimal

Salaire total : **\$ 400.0**

Apprentissage et analyse des résultats

PuLP permet de faire automatiquement la résolution de problèmes linéaires.

Dans notre exercice, il était facile de simuler ce traitement avec deux boucles imbriquées mais j'imagine que cet outil permet d'exécuter des algorithmes beaucoup plus complexes avec la même facilité.

Temps algorithmique (Grand O)

1- Trois boucles imbriquées une à l'intérieur de l'autre

Réponse : $O(n_1 n_2 n_3)$ où n_1 = le nombre d'itération de la première boucle, n_2 le nombre d'itérations de la seconde boucle et n_3 est le nombre d'itérations de la troisième boucle.

$$\text{Nombre d'itérations total} = n_1 * n_2 * n_3$$

Utile pour parcourir un tableau en trois dimensions au complet.

Temps de traitement : Plutôt lent.

2- Une variable directement utilisée

Réponse $O(1)$

Très simple car donne un accès direct à la donnée.

Temps de traitement : Instantané.

3- Une recherche dans un arbre binaire

Réponse : $O(\log n)$ pour un arbre équilibré

Temps de traitement : Proportionnel à la hauteur de l'arbre(n)

Récurtivité Simple

Créer une fonction récursive nommée `add_recursive` qui additnonne un nombre « n » avec lui-même en faisant des sauts de moins 1.

Code Python :

```
def add_recursive(nombre):  
    if (nombre == 1):  
        return 1  
    else:  
        return add_recursive(nombre - 1) + add_recursive(nombre - 1)  
  
if __name__ == '__main__':  
    print("Nombre = 4 :", add_recursive(4))  
    print("Nombre = 5 :", add_recursive(5))  
    print("Nombre = 10:", add_recursive(10))  
    print("Nombre = 30:", add_recursive(30))
```

Résultats:

nombre= 4 : 8

nombre = 5 : 16

nombre = 10: 512

nombre = 30: 536870912

Notions d'apprentissage et élaboration des résultats :

La programmation récursive permet de briser en problèmes plus simples un problème complexe. Chaque itération simplifie le problème jusqu'à ce qu'on atteigne une valeur connue (dans notre cas on arrête et dépile lorsque le nombre =1) ou que l'on détecte une anomalie qui fait que l'on ne veut plus que le programme poursuive.

Chaque itération résolue est ajoutée rétroactivement en dépilant et lorsque le programme revient à sa première itération le problème est résolu.

Ceci permet de faire des calculs vraiment complexes en très peu de ligne de code.

Programmation Dynamique

Faire l'exercice précédent avec la programmation dynamique avec mémorisation.

Pour y arriver nous allons sauvegarder les données dans un dictionnaire et réutiliser les valeurs qui ont déjà été calculées.

Code Python :

```
def add_recursive_mem(nombre:int , memo={}):  
    if (nombre == 1):  
        return 1  
    else:  
        if nombre in memo:  
            return memo[nombre]  
        else:  
            memo[nombre] = add_recursive_mem(nombre - 1, memo) +  
add_recursive_mem(nombre - 1, memo)  
            return memo[nombre]  
  
if __name__ == '__main__':  
    print("Nombre = 4 :", add_recursive_mem(4))  
    print("Nombre = 5 :", add_recursive_mem(5))  
    print("Nombre = 10:", add_recursive_mem(10))  
    print("Nombre = 30:", add_recursive_mem(30))
```

Résultats :

nombre= 4 : 8

nombre = 5 : 16

nombre = 10: 512

nombre = 30: 536870912

Analyse des résultats :

On obtient les mêmes résultats que précédemment sauf que ça semble rouler beaucoup plus vite lorsque le nombre est élevé.

Le principe est que l'algorithme récursif se répète jusqu'à ce que la valeur =1. En ajoutant un dictionnaire avec les valeurs déjà calculée au paramètres, le programme retourne

directement cette valeur au lieu de refaire l'arbre de calcul au complet. Chaque nombre plus petit que le nombre choisi est calculé une seule fois plutôt que 2^n fois ou $n =$ le nombre passé en paramètre.

Test de Performance

Exécution des deux algorithmes de récursivité et de programmation dynamique et comparaison du temps de traitement.

Code Python

```
from Programmation_dynamique import *
from Récursivité_simple import *
import time

def performance(nombre):
    #Exécution de récursivité simple
    heure_début_simple = time.perf_counter() # calcul de l'heure du début
    print('Simple: ', str(add_recursive(nombre))) # Impression du résultat du
calcul simple
    heure_fin_simple = time.perf_counter() # calcul de l'heure de fin

    #Exécution de récursivité complexe
    heure_début_dynamique = time.perf_counter()
    print('Dynamique: ', str(add_recursive_mem(nombre))) #impression du
résultat du calcul dynamique
    heure_fin_dynamique = time.perf_counter()

    #calcul des temps d'exécution
    runtime_simple= heure_fin_simple - heure_début_simple # calcul du temps
de traitement récursif
    runtime_dynamique = heure_fin_dynamique - heure_début_dynamique # Calcul
du temps réponse dynamique

    #Impression des temps de traitement
    print('simple(' + str(nombre) + f') {runtime_simple:.3f} secondes')
    print('dynamique(' + str(nombre) + f') {runtime_dynamique:.3f} secondes')

if __name__ == '__main__':
    performance(30)
```

Résultat

Cours IA-1/PythonProject/Test de performance.py

Simple: 536870912

Dynamique: 536870912

simple(30) 37.478 secondes

dynamique(30) 0.000 secondes

Process finished with exit code 0

Analyse des résultats

Le temps réponse varie exponentiellement entre l'algorithme de Récursivité Simple qui boucle 536870912 fois (2^{29}) la programmation dynamique qui, elle ne boucle que 30 fois.

La programmation dynamique est beaucoup plus efficace lorsque possible.